**ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ**

МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ




# БИЕ ДААЛТ 1
# ТАЙЛАН

## Үйлдлийн систем (F.CSM302)

**Сэдэв:** Multithreaded Sorting Application, Sorting animation

**Хуваарилалтын код:** II-4-6

**Хичээл заасан багш:** Г. Ганбат /F.CS10/

**Гүйцэтгэсэн оюутнууд:** Б. Отгондалай /B241940132/
М. Мөнхгэрэлт /B232270012/
А. Базарсад /B242270065/
Б. Хаш-Эрдэнэ /B241930006/
Э. Мичидмаа /B221960015/
Ж. Суварнабазар /B251940118/

Улаанбаатар хот, 2026

# Хураангуй

**Асуудал.** Олон цөмтэй компьютерыг ашиглахын тулд ажлыг thread-үүдэд хуваах шаардлагатай. Гэхдээ хуваасан ажлын хэсгүүд хэн ямар өгөгдөлд бичих, хэзээ дуусахыг хэрхэн хүлээх, үр дүнг хэрхэн нэгтгэхийг зөв зохицуулахгүй бол үр дүн буруу гарна. *Operating System Concepts* (10-р хэвлэл) номын 4-р бүлгийн “Multithreaded Sorting Application” төсөл энэ асуудлыг эрэмбэлэлтийн жишээн дээр судалдаг.

**Шийдэл.** Java 17 дээр жагсаалтыг хоёр хуваан, хоёр *sorting thread* тус бүр параметрээр өгсөн мужаа эрэмбэлж, гурав дахь *merging thread* эх массивтай ижил хэмжээтэй хоёр дахь global массив руу нэгтгэдэг програм хийв. Parent thread бүх thread-ийг `join()` хийсний дараа үр дүнг хэвлэнэ. Синхрончлолыг бүхэлд нь `start()`/`join()` хангадаг тул lock шаардлагагүй. Таван алгоритм (insertion, bubble, selection, merge, quick), оролтын шалгалт, JSON event trace хийсэн. Мөн бодит thread-тэй холбогдсон хөтчийн *Sorting animation* хийсэн бөгөөд хоёр горимтой: бүртгэсэн алхмыг тоглуулах *Replay*, амьд thread-ийг `Semaphore`-оор нэг үйлдлээр урагшлуулах *Lockstep*.

**Туршилт.** JUnit-гүй 100 автомат тест бичиж, macOS (Temurin 17) ба Linux (OpenJDK 21) орчинд ажиллуулав. Тест нь номын жишээ, захын тохиолдол, буруу оролт, давхардал, $n = 10^6$ хүртэлх том өгөгдөл, thread-ийн дараалал, trace, Lockstep-ийг хамарна. Хугацааг 1 thread, 1 thread хуваж, 3 thread гэсэн гурван хувилбараар, 3 тусдаа JVM дээр хэмжив.

**Гол үр дүн.** 100/100 тест амжилттай. Sorter бүр зөвхөн өөрийн мужид бичдэгийг trace-ээс баталсан. Хоёр цөмтэй машин дээр merge ба quick sort-ын хурдсалт $n = 10^6$ үед $1.7\times$ орчим гарсан. Энэ нь 2-оос бага бөгөөд шалтгаан нь нэгтгэх үе дан thread-ээр явдагт оршино (Амдалын хууль). O($n^2$) алгоритмд ($n = 20\,000$) insertion ба selection sort-ын хурдсалт $2.4$–$3.6\times$ гарсан ч ашгийн ихэнх нь массивыг хуваасантай холбоотой, учир нь хоёр хэсгийг эрэмбэлэх ажил $2(n/2)^2 = n^2/2$ байна. Жижиг $n$-д (O($n^2$) алгоритмд $n \le 1000$, merge ба quick sort-д $n \le 10^4$) thread үүсгэх, `join()` хийх зардал давамгайлж, олон урсгалт хувилбар нэг урсгалтаас удаан ажилласан.

**Түлхүүр үг:** thread, олон урсгалт програмчлал, `join`, зэрэгцээ тооцоолол, эрэмбэлэлт, merge, semaphore, Амдалын хууль, Java.

# Агуулга

**Хураангуй** . . . . . **i**

**1 Удиртгал** . . . . . **1**
1.1 Зорилго . . . . . 1
1.2 Ач холбогдол . . . . . 1
1.3 Хамрах хүрээ . . . . . 1

**2 Сурах бичгийн шаардлага** . . . . . **1**
2.1 Номын үндсэн шаардлага . . . . . 1
2.2 Нэмэлт шаардлага . . . . . 2
2.3 Шаардлагын хангалт . . . . . 2
2.4 Номын жишээ ба сондгой $n$ . . . . . 2

**3 Онолын үндэс** . . . . . **3**
3.1 Thread ба зэрэгцээ ажиллагаа . . . . . 3
3.2 Амдалын хууль . . . . . 3
3.3 Java дахь thread ба синхрончлол . . . . . 3
3.4 Эрэмбэлэх алгоритмууд . . . . . 4

**4 Системийн зохиомж** . . . . . **4**
4.1 Бүрэлдэхүүн хэсгүүд . . . . . 4
4.2 Өгөгдөл ба урсгал . . . . . 5
4.3 Синхрончлол: яагаад lock хэрэггүй вэ . . . . . 6
4.4 Lockstep горим . . . . . 6
4.5 Хязгаар ба хамгаалалт . . . . . 6

**5 Хэрэгжилт** . . . . . **7**
5.1 Хэл, сан, орчин . . . . . 7
5.2 Parent, sorting ба merging thread . . . . . 7
5.3 Ролт ба event trace . . . . . 8
5.4 GitHub ба эх код . . . . . 8

**6 Туршилт ба UI дүрслэл** . . . . . **8**
6.1 Зарчим . . . . . 8
6.2 Өөрчилж турших параметр ба удирдлага . . . . . 9

6.3 Юуг ажиглах . . . . . 9
6.4 Replay ба Lockstep горим . . . . . 9

**7 Туршилтын арга . . . . . 10**
7.1 Орчин . . . . . 10
7.2 Автомат тест . . . . . 10
7.3 Хугацааны хэмжилтийн арга . . . . . 11

**8 Үр дүн . . . . . 12**
8.1 Тестийн үр дүн . . . . . 12
8.2 Хугацааны хэмжилтийн үр дүн . . . . . 12
8.3 Илэрсэн ба зассан алдаа . . . . . 13

**9 Хэлэлцүүлэг . . . . . 14**
9.1 Үр дүнгийн тайлбар . . . . . 14
9.2 Хязгаарлалт . . . . . 14
9.3 Сайжруулах боломж . . . . . 15
9.4 GitHub ба AI ашигласан тухай . . . . . 15

**10 Дүгнэлт . . . . . 15**

**Ашигласан материал . . . . . 16**

# 1. Удиртгал

## 1.1. Зорилго

*Operating System Concepts* номын 4-р бүлгийн “Multithreaded Sorting Application” төслийг Java 17 дээр хэрэгжүүлэх нь ажлын зорилго юм. Жагсаалтыг хоёр хуваан хоёр sorting thread-ээр эрэмбэлж, гурав дахь merging thread-ээр хоёр дахь global массив руу нэгтгэнэ. Thread-ийн бодит ажиллагааг хөтөч дээр *Sorting animation* хэлбэрээр харуулна. Мөн оролтын шалгалт, автомат тест хийж, олон урсгалт шийдэл хэзээ ашигтай болохыг хугацааны хэмжилтээр шинжилнэ.

## 1.2. Ач холбогдол

Орчин үеийн компьютер олон цөмтэй боловч нэг урсгалтай програм тэдгээрийн зөвхөн нэгийг ашигладаг. Ажлыг thread-үүдэд хуваахад хэн ямар өгөгдөлд бичих, хэзээ дуусахыг хүлээх, үр дүнг хэрхэн нэгтгэх гэсэн асуудал гарч ирдэг [1]. Энэ төсөл нь *өгөгдлийн зэрэгцээ* (data parallelism) болон *fork–join* загварыг энгийн жишээн дээр харуулдаг.

## 1.3. Хамрах хүрээ

Програм нь Java 17-ийн зөвхөн стандарт санг ашиглана. Командын мөр (CLI) болон локал web сервер дээрх хөтчийн UI гэсэн хоёр интерфейстэй. Утгууд `int` мужид байх ба хэмжээний хязгаарыг 4.5-р хэсэгт өгөв. Эх код, тест, гарын авлага GitHub repository-д байгаа [10].

# 2. Сурах бичгийн шаардлага

## 2.1. Номын үндсэн шаардлага

Бүхэл тоон жагсаалтыг ижил хэмжээтэй хоёр хэсэгт хуваж, хоёр тусдаа thread тус бүр нэг хэсгийг дурын алгоритмаар эрэмбэлнэ. Гурав дахь thread хоёр хэсгийг эх массивтай ижил хэмжээтэй хоёр дахь global массив руу нэгтгэнэ. Parent thread бүх sorting thread дуусахыг хүлээж, дараа нь merging thread-ийг эхлүүлж, түүнийг дуустал хүлээгээд үр дүнг хэвлэнэ. Thread бүрд эрэмбэлэх хэсгийн эхлэх индекс зэрэг параметрийг дамжуулна [1, Ch. 4, Programming Projects].

## 2.2. Нэмэлт шаардлага

Сэдвийн хуваарилалтаар *Sorting animation* нэмэгдсэн: дүрслэл нь бодит thread-тэй холбоотой байж, хэмжээ ба утгыг өөрчлөх, Run / Step / Pause / Reset, хурдны удирдлагатай байна. Мөн багшийн зааврааp гарын авлага, тайлан, GitHub repository шаардлагатай. Баг өөрөө оролтын бүрэн шалгалт (хоосон, тоо биш, хэт том утга, $n = 0 \dots 3$, давхардал, сондгой $n$) болон зөвхөн стандарт сан ашиглах шаардлагыг тавьсан.

## 2.3. Шаардлагын хангалт

Хүснэгт 2.1. Номын шаардлага ба хэрэгжүүлэлт

| Шаардлага | Хэрэгжүүлэлт |
|---|---|
| Жагсаалтыг хоёр хуваах | `splitIndex(n)` $= \lceil n/2 \rceil$; зүүн `[0, mid)`, баруун `[mid, n)` |
| Хоёр sorting thread | `SortingThread extends Thread`; 5 алгоритмаас хэсэг тус бүрд сонгоно |
| Global жагсаалт ба хоёр дахь массив | `SharedData.list`, `SharedData.result = new int[n]` |
| Merging thread | `MergingThread`: хоёр заагчтай, тогтвортой нэгтгэл |
| Parent хүлээгээд нэгтгэнэ | `left.join(); right.join(); merger.start(); merger.join();` |
| Параметр дамжуулах | `SortParams(name, startIndex, endIndex, algorithm)`, `MergeParams` |
| Үр дүн хэвлэх | Үр дүн, thread бүрийн статистик, `Arrays.sort`-той харьцуулсан шалгалт |

## 2.4. Номын жишээ ба сондгой $n$

Номын жишээ `7, 12, 19, 3, 18, 4, 2, 6, 15, 8`-ийн хоёр хэсэг `3 7 12 18 19 | 2 4 6 8 15` болж эрэмбэлэгдээд, `2 3 4 6 7 8 12 15 18 19` гэж нэгтгэгдэнэ. Номд хэсгүүдийг “ижил хэмжээтэй” гэсэн ч сондгой $n$ үед энэ боломжгүй. Бид сондгой $n$-ийг алдаа гэж үзэхийн оронд хэсгүүдийг $\lceil n/2 \rceil$ ба $\lfloor n/2 \rfloor$ болгосон. Нэгтгэх алгоритм тэнцүү хэмжээ шаарддаггүй бөгөөд ялгаа нь ердөө 1 элемент юм. $n = 1$ үед баруун хэсэг хоосон байх ч номын бүтцийг хадгалж Sorter-R thread ажиллаад шууд дуусна.

# 3. Онолын үндэс

## 3.1. Thread ба зэрэгцээ ажиллагаа

Thread бол CPU-г ашиглах үндсэн нэгж бөгөөд өөрийн регистр, программ тоолуур, stack-тай. Нэг процессын thread-үүд code, data хэсэг болон нээлттэй файлаа хуваалцдаг [1, 4.1]. Иймээс манай хоёр sorting thread ба merging thread нэг процессын хоёр массивт шууд ханддаг. *Concurrency* нь хэд хэдэн ажил зэрэг урагшлахыг, *parallelism* нь тэдгээр бодитоор өөр өөр цөм дээр нэгэн агшинд гүйцэтгэгдэхийг хэлнэ [1, 4.2]. Хоёр sorter ижил үйлдлийг өгөгдлийн хоёр хэсэг дээр хийдэг тул энэ нь *өгөгдлийн зэрэгцээ* юм. Parent thread хүүхэд thread-үүдийг үүсгэж (fork), дуусахыг нь хүлээж (join), дараа нь нэгтгэдэг загварыг *fork–join* гэнэ [1, 4.5.2].

## 3.2. Амдалын хууль

Програмын дараалсан хэсгийн эзлэх хувийг $S$, цөмийн тоог $N$ гэвэл хамгийн их хурдсалт [1, 4.2], [2]

$$\text{speedup} \leq \frac{1}{S + (1 - S)/N}. \tag{3.1}$$

Манай төсөлд нэгтгэх үе дан thread-ээр явдаг тул $S > 0$ бөгөөд $N = 2$ үед ч хурдсалт 2-т хүрэхгүй.

## 3.3. Java дахь thread ба синхрончлол

Java-д thread-ийг `java.lang.Thread`-ээр төлөөлнө. `start()` нь шинэ native thread үүсгэж `run()`-ийг тэнд ажиллуулна, `join()` нь тухайн thread дуустал хүлээнэ. Энэ нь Pthreads-ийн `pthread_create`, `pthread_join`-той дүйцнэ [1, 4.4]. Thread-ийн төлөвийг `Thread.getState()` буцаана (`NEW`, `RUNNABLE`, `WAITING`, `TERMINATED` гэх мэт) [4].

Java Memory Model-ийн *happens-before* дүрмээр `t.start()`-ийн өмнө parent-ийн хийсэн бүх зүйл thread `t`-д харагдана. Мөн `t`-ийн хийсэн бүх зүйл `t.join()` буцсаны дараа parent-д харагдана [3, 7]. Энэ хоёр дүрэм манай шийдлийн зөв байдлын үндэс болдог (4.3-р хэсэг).

Semaphore нь `acquire()` (тоолуур 0 бол хүлээнэ) ба `release()` гэсэн хоёр атомар үйлдэлтэй тоолуур юм [1, 6.6]. `new Semaphore(0, true)` гэж

*fair* горимоор үүсгэвэл хамгийн удаан хүлээсэн thread түрүүлж permit авна [5].

## 3.4. Эрэмбэлэх алгоритмууд

Хүснэгт 3.1. Хэрэгжүүлсэн алгоритмууд [8, 9]

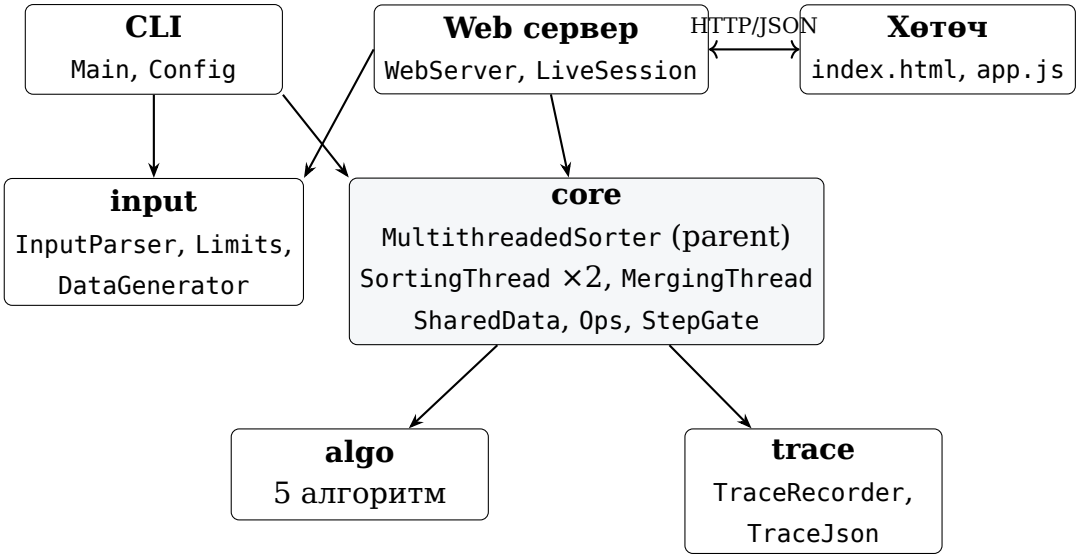
| Алгоритм | Дундаж | Муу | Онцлог |
|---|---|---|---|
| Insertion | $O(n^2)$ | $O(n^2)$ | Хөрш элементүүдийг сольж байрлуулна |
| Bubble | $O(n^2)$ | $O(n^2)$ | Солилт хийгээгүй бол эрт зогсоно |
| Selection | $O(n^2)$ | $O(n^2)$ | Хамгийн багыг олж нэг солилтоор байрлуулна |
| Merge | $O(n \log n)$ | $O(n \log n)$ | Top-down, туслах буфертэй, тогтвортой |
| Quick | $O(n \log n)$ | $O(n^2)$ | Hoare partition, дунд pivot, stack $O(\log n)$ |

Хоёр эрэмбэлэгдсэн хэсгийг нэгтгэх нь хоёр заагчтай $O(n)$ алгоритм бөгөөд тэнцүү үед зүүн хэсгийнхийг авснаар тогтвортой болно [8].

# 4. Системийн зохиомж

## 4.1. Бүрэлдэхүүн хэсгүүд

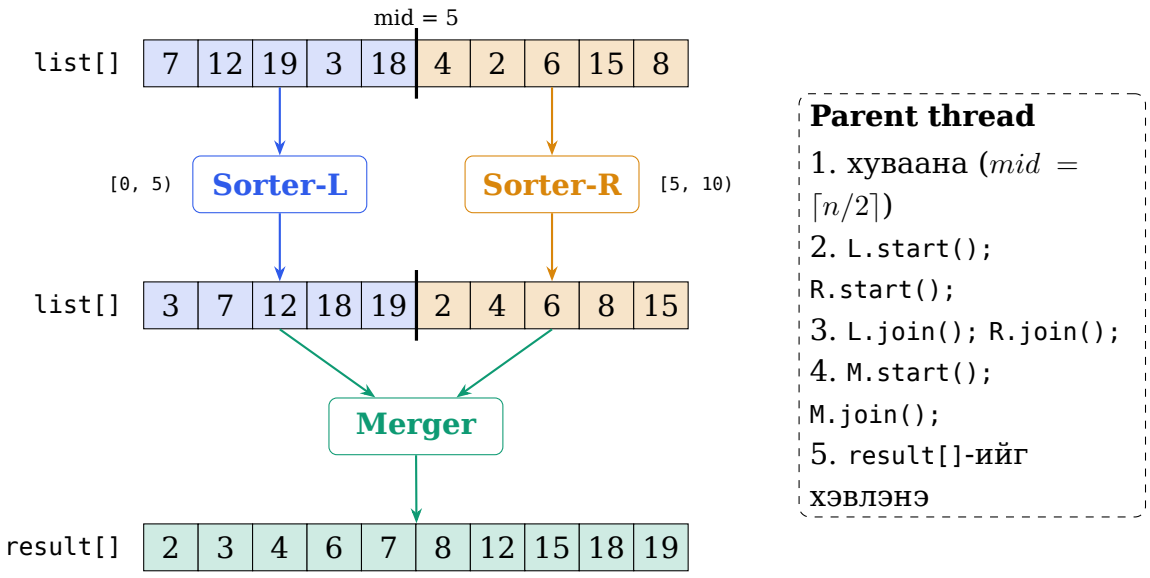
Програмын төв нь `core` багц бөгөөд номын бүтцийг (parent, хоёр sorter, merger) хэрэгжүүлдэг (Зураг 4.1). CLI ба web сервер хоёулаа энэ нэг цөмийг дууддаг. Тиймээс UI дээр харагдаж буй thread-үүд нь командын мөрийн thread-үүдтэй яг ижил код юм. Бусад багцын үүрэг: `algo` (5 алгоритм), `input` (оролт задлах, хязгаар, санамсаргүй өгөгдөл), `trace` (event бүртгэх, JSON), `web` (HTTP сервер, Lockstep session). Нийт ойролцоогоор 2 200 мөр Java код байна.




Зураг 4.1. Програмын бүтэц. Сум нь “ашигладаг” харьцааг заана

## 4.2. Өгөгдөл ба урсгал

`SharedData` нь хоёр “global” массивтай: `list` (sorter-ууд хэсгээ энд эрэмбэлнэ) ба ижил хэмжээтэй `result` (merger энд бичнэ). Thread бүр `SortParams` (нэр, `startIndex`, `endIndex`, алгоритм) эсвэл `MergeParams` (`leftStart`, `mid`, `end`) параметрийг авна. Номын жишээн дээрх урсгалыг Зураг 4.2-т үзүүлэв.




Зураг 4.2. Номын жишээн дээрх өгөгдлийн урсгал

## 4.3. Синхрончлол: яагаад lock хэрэггүй вэ

Sorter-L зөвхөн `[0, mid)`, Sorter-R зөвхөн `[mid, n)` мужид бичдэг тул тэдгээрийн хооронд race condition үүсэхгүй. Merger нь `list[]`-ийг уншдаг боловч parent хоёр sorter-ийг `join()` хийсний *дараа* л `start()` хийдэг. Happens-before дүрмээр sorter-уудын бүх бичилт merger-д заавал харагдана. Parent мөн `result[]`-ийг merger-ийг join хийсний дараа уншина. Ингэснээр синхрончлолыг бүхэлд нь `start()`/`join()` хангаж, lock огт хэрэггүй болно. Үүнийг тест trace-ээс баталсан (7-р бүлэг). Цорын ганц тусгай синхрончлол нь event бүртгэл (lock-гүй `ConcurrentLinkedQueue`, `AtomicLong`) ба Lockstep горимд байдаг.

## 4.4. Lockstep горим

Lockstep горимд sorting ба merging thread бүр compare, swap, write үйлдэл бүрийн өмнө fair `Semaphore`-оос нэг permit авна. Permit байхгүй үед thread бодитоор `WAITING` төлөвт зогсоно. UI-ийн “Алхам” товч серверт `release(1)` хийлгэж, нэг thread яг нэг үйлдэл хийнэ. FIFO дарааллаас болж хоёр sorter ээлжилнэ (L, R, L, R). Харин эхний permit-ийг аль sorter авах нь аль нь gate-д түрүүлж хүрэхээс хамаарна. Session-ийг цуцлахад бүх хүлээж буй thread сэрж гарна.

## 4.5. Хязгаар ба хамгаалалт

Хүснэгт 4.1. Оролтын хязгаар

| Зүйл | Хязгаар | Шалтгаан |
|---|---|---|
| Утга | `int` муж | $-2^{31} \dots 2^{31}-1$ |
| O($n^2$) алгоритм | $n \le 20\,000$ (`--force`) | Удаан ажиллахаас сэргийлнэ |
| O($n \log n$) алгоритм | $n \le 10^6$ | Санах ой, хугацаа |
| Event trace | $n \le 1000$ / $10\,000$ | O($n^2$) / merge, quick. Файл хэт томрохоос сэргийлнэ |
| Animation (Replay / Lockstep) | $n \le 300$ / $100$ | Уншигдахуйц байх, серверийн ачаалал |

Web сервер зөвхөн `127.0.0.1` дээр сонсож, зэрэг 4 хүртэл Lockstep session зөвшөөрнө. 10 минут хөдөлгөөнгүй session автоматаар

цуцлагдана.

# 5. Хэрэгжилт

## 5.1. Хэл, сан, орчин

Java 17 (`javac --release 17`) дээр зөвхөн стандарт сан ашиглан бичсэн. Thread-д `java.lang.Thread`, Lockstep-д `java.util.concurrent.Semaphore`, web серверт JDK-д багтсан `com.sun.net.httpserver`, UI-д framework-гүй HTML/CSS/JavaScript (`<canvas>`) ашигласан. Maven, Gradle, JUnit хэрэглээгүй. Build, ажиллуулах, тест, хэмжилтийг `build.sh`, `run.sh`, `test.sh`, `bench.sh` скриптүүд (Windows: `.bat`) хийнэ.

## 5.2. Parent, sorting ба merging thread

Parent-ийн логик (Код 5.1) номын тайлбарыг шууд дагадаг: хуваах, хоёр sorter-ийг эхлүүлэх, хоёуланг нь хүлээх, merger-ийг эхлүүлэх, хүлээх.

```
int mid = (n + 1) / 2;                          // ceil(n/2)
SharedData data = new SharedData(input);        // list[] and result[]
SortingThread left  = new SortingThread(data, new SortParams("Sorter-L", 0, mid, la,
    leftAlgo), ...);
SortingThread right = new SortingThread(data, new SortParams("Sorter-R", mid, n, ra,
    rightAlgo), ...);
left.start();  right.start();
left.join();   right.join();
MergingThread merger = new MergingThread(data, new MergeParams("Merger", 0, mid, n),
    ...);
merger.start();
merger.join();
return new SortResult(input.clone(), data.result, mid, left, right, merger, sortPhase,
    total);
```

Код 5.1. Parent thread (MultithreadedSorter.sort, trace ба gate-ийн мөрийг товчлов)

`SortingThread.run()` нь параметрээр авсан `[startIndex, endIndex)` мужийг сонгосон алгоритмаар эрэмбэлнэ. `MergingThread` нь зүүн хэсгийн заагч `i`, баруун хэсгийн заагч `j`-ийг харьцуулж багыг нь `result[k]`-д бичнэ (тэнцүү үед зүүнийхийг авна). Thread дотор алдаа гарвал тэр нь parent руу автоматаар дамжихгүй тул thread бүр алдааг хадгалж, parent join хийсний дараа шалгадаг.

Алгоритмууд массивт шууд хандахын оронд `Ops` объектын `compare`, `swap`, `set` аргуудыг дууддаг. Thread бүр өөрийн `Ops`-той тул үйлдлийн тоолуурт lock хэрэггүй. Trace асаалттай бол үйлдэл бүрийг event

болгон бүртгэж, Lockstep горимд үйлдэл бүрийн өмнө permit хүлээнэ. Хоёулаа унтраалттай үед энэ нь ердийн массивын хандалт болно.

## 5.3. Оролт ба event trace

`InputParser` нь таслал, зай, шинэ мөрөөр тусгаарласан утга бүрийг бүхэл, бутархай, тоо биш гэж ангилж, `int` мужийг шалгана. Алдааны мессеж байрлалыг нэрлэнэ (жишээ: “3-р утга ’abc’ нь бүхэл тоо биш.”). `Limits` нь 4.5-р хэсгийн хязгаарыг шалгаж, сондгой $n$-ийн үед тэмдэглэл гаргана. Оролтын алдаанд CLI exit code 2, амжилттай үед 0 буцаана.

`TraceRecorder` нь event бүрт `AtomicLong`-оос дэс дугаар өгч, lock-гүй дараалалд нэмнэ. Event нь `{seq, t, th, type, a, b, c}` бүтэцтэй (`CMP`, `SWP`, `SET`, `MCMP`, `MW`, `START`, `JOIN` гэх мэт) бөгөөд `--trace FILE`-аар JSON болгон хадгална. Хоосон талбарын тэмдэглэгээг эхэндээ `Integer.MIN_VALUE` гэж сонгосон нь бодит утгатай давхцаж байсан тул `long` талбар ба `Long.MIN_VALUE` болгож зассан.

## 5.4. GitHub ба эх код

Эх код `https://github.com/Munkhgerelt/F.CSM302-TeamProject-2-4-6` хаягт байрладаг. Ажлыг үе шатад хувааж (core, алгоритм, оролт, trace, web UI, Lockstep, тест, хэмжилт, баримт бичиг), үе шат бүрийн төгсгөлд зөвхөн хэрэгтэй файлыг сонгон Монгол мессежтэй commit хийсэн. Эцсийн хувилбарыг `v1.0` tag бүхий GitHub Release болгон нийтэлсэн. Build-ийн файлуудыг `.gitignore`-оор хассан. Repository-д `src/` (код), `tests/` (тест, хэмжилт, үр дүнгийн хүснэгт), `examples/` (тестийн өгөгдөл), `docs/` (гарын авлага), `report/` (тайлан) хавтас байна.

# 6. Туршилт ба UI дүрслэл

## 6.1. Зарчим

Animation-ийн гол зарчим нь дэлгэц дээрх алхам бүр бодит thread-ийн хийсэн үйлдэл байх явдал юм. Хөтөч эрэмбэлэлтийг өөрөө тооцоолдоггүй. Java дээрх thread-үүдийн бүртгэсэн event-үүдийг хүлээн авч дүрсэлнэ. Event бүрт thread-ийн нэр ба `System.nanoTime`-аар хэмжсэн хугацаа байдаг тул хоёр sorter-ийн алхмууд хэрхэн холилдсон, merger хэзээ эхэлснийг бодитоор нь харуулна. `./run.sh --web` ажиллуулаад `http://localhost:8080` хаягийг нээнэ.

## 6.2. Өөрчилж турших параметр ба удирдлага

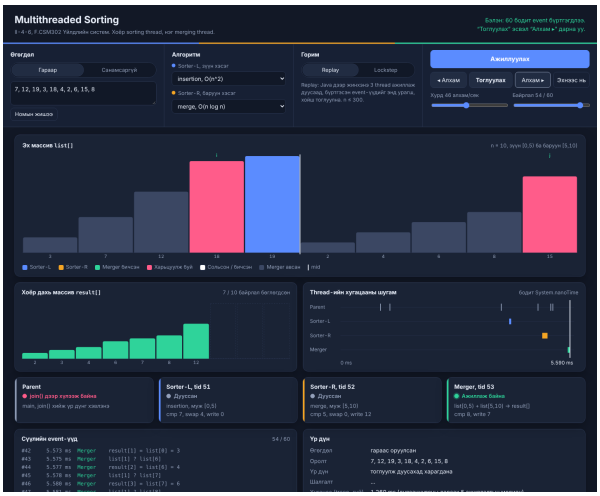
Хэрэглэгч тоонуудаа гараар оруулах эсвэл санамсаргүй жагсаалт үүсгэнэ ($n$, seed, min, max, хэлбэр: random, sorted, reversed, duplicates). Sorter-L ба Sorter-R-д таван алгоритмаас тус тусад нь сонгоно. Удирдлага нь Ажиллуулах, Тоглуулах / Түр зогсоох (Space), Алхам урагш (→) ба хойш (←), Эхнээс нь, хурд (1–5000 алхам/сек), байрлалын гулсуур гэсэн товчнуудтай.

## 6.3. Юуг ажиглах

Дэлгэц эх массив list[], хоёр дахь массив result[], thread-үүдийн хугацааны шугам, thread бүрийн карт (нэр, tid, төлөв, үйлдлийн тоо), сүүлийн event-үүд, үр дүн гэсэн хэсгүүдтэй (Зураг 6.1). Цэнхэр нь Sorter-L-ийн хэсэг, улбар шар нь Sorter-R-ийн хэсэг, ногоон нь merger-ийн бичсэн утга, ягаан улаан нь харьцуулж буй элемент, цагаан нь сольсон элемент, бараан саарал нь merger аль хэдийн авсан элемент. Босоо зураас хуваалтын цэг mid-ийг заана.




(a) Sorter-L insertion sort-оор эрэмбэлж байна




(b) Merger result[] руу нэгтгэж байна

Зураг 6.1. Replay горим, номын жишээ

## 6.4. Replay ба Lockstep горим

**Replay** горимд сервер бодит 3 thread-ийг ажиллуулж, бүх event-ийг хөтөч рүү буцаана. Хөтөч эхний $k$ event-ийг эх жагсаалт дээр хэрэглэж дурын алхамд очих тул урагш, хойш явах боломжтой. Мөн trace-гүй ажиллалтын хугацааг тусад нь харуулна.

**Lockstep** горимд thread-үүд амьд хэвээр байж, үйлдэл бүрийн өмнө Semaphore дээр зогсоно. “Алхам” бүр серверт нэг permit өгч, шинэ event-үүд болон thread бүрийн `Thread.getState()`-ийг авна (Зураг 6.2). Бодит thread-ийг буцааж болохгүй тул энэ горимд хойш алхах боломжгүй. Хугацааны тэмдэглэгээ нь хэрэглэгчийн товч дарах хоорондын хугацааг агуулдаг тул хурд хэмжихэд Replay горимыг ашиглана.




Зураг 6.2. Lockstep: хоёр sorter Semaphore дээр `WAITING`, parent `join()` дээр `WAITING`

# 7. Туршилтын арга

## 7.1. Орчин

Тестийг багийн гишүүний компьютер (macOS 27, Apple Silicon, 10 цөм, Eclipse Temurin 17.0.21) болон хөгжүүлэлтийн Linux контейнер (Ubuntu 24.04, Intel Xeon, 2 цөм, OpenJDK 21) дээр ажиллуулав. Хугацааны хэмжилтийг контейнер дээр хийсэн.

## 7.2. Автомат тест

Тестийг JUnit-гүй, `tests/TestRunner.java` файлд бичсэн бөгөөд `./test.sh` ажиллуулна. Тест бүрд урьдчилан бичсэн хүлээгдсэн үр дүн

ба програмын бодит үр дүнг харьцуулж, бүх эрэмбэлэлтийг `Arrays.sort`-той тулгана. Үр дүнгийн хүснэгт `tests/results/test-results.md`-д бичигдэнэ.

Хүснэгт 7.1. Автомат тестийн ангилал

| Ангилал | Тоо | Шалгах зүйл |
|---|---|---|
| Номын жишээ | 26 | 25 алгоритмын хослол, mid = 5, хоёр хэсэг |
| Захын тохиолдол | 17 | $n = 1, 2, 3$, бүгд ижил, сөрөг, INT_MIN/MAX, эрэмбэтэй, урвуу, $\lceil n/2 \rceil$ / $\lfloor n/2 \rfloor$ |
| Буруу оролт | 18 | Хоосон, тоо биш, бутархай, int-ээс гадуур, хязгаарууд, буруу файл |
| Давхардал ба хэлбэр | 6 | 4 хэлбэр × 25 хослол, 2000 санамсаргүй ажиллалт |
| Том өгөгдөл | 4 | $n = 10^6$ (merge, quick), $n = 20\,000$ (O($n^2$)) |
| Thread | 5 | 3 өөр thread, параметр, `TERMINATED`, дараалал, муж зөрчөөгүй |
| Event trace | 7 | Trace-ээс сэргээсэн `result[]` = бодит, JSON формат |
| Lockstep | 5 | `WAITING`, 1 permit = 1 үйлдэл, ээлжлэх, цуцлах |
| Командын мөр | 12 | Тусдаа JVM дээр гаралт ба exit code |

Тест нь зөвхөн үр дүнг биш, номын дотоод нөхцлийг ч шалгадаг. Trace-ийн бүх event-ээр Sorter-L нь `[0, mid)`-ээс, Sorter-R нь `[mid, n)`-ээс гадуур хандаагүйг баталсан. Мөн merger хоёр sorter дууссаны дараа эхэлснийг 100 удаа шалгасан. Lockstep-ийн тестийг 30 удаа дараалан болон нэг цөм дээр (`taskset -c 0`) ажиллуулсан. Web UI-г хөгжүүлэлтийн явцад Playwright (Chromium)-оор дэлгэцийн 1, 1.5, 2 масштабтай автоматаар шалгаж, JavaScript алдаа 0 байгааг баталсан.

## 7.3. Хугацааны хэмжилтийн арга

`tests/Benchmark.java` (`./bench.sh`) нь гурван хувилбарыг харьцуулна: *1 thread* (нэг thread бүх массивыг эрэмбэлнэ), *1 thread, хувааж* (номын бүтэцтэй ижил ажлыг нэг thread дараалан хийнэ) ба *3 thread* (номын бүтэц). Нийт хурдсалт $= T_1/T_3$, зэрэгцээний хурдсалт $= T_{1,\text{хувааж}}/T_3$. Хоёр дахь нь хуваалтын нөлөөг зэрэгцээний нөлөөнөөс салгана. Оролт санамсаргүй `int`, trace унтраалттай. JIT-ийн нөлөөг багасгахын тулд 3 тусдаа JVM ажиллуулж, тус бүрд 3 удаа дулаацуулсны дараа 9 удаа хэмжиж, медианы медианыг авсан.

# 8. Үр дүн

## 8.1. Тестийн үр дүн

100 автомат тест хоёр орчинд хоёуланд нь амжилттай (100/100) болсон. Lockstep-ийн тестийг 30 удаа давтаж, нэг цөм дээр ажиллуулахад ч алдаа гараагүй. Тест бүрийн оролт, хүлээгдсэн ба бодит утгыг `tests/results/test-results.md`-д өгөв.

```
== Lockstep ==
  [OK  ]  84. Permit 0 үед sorter-ууд WAITING
  [OK  ]  85. 1 permit = яг 1 үйлдэл
  [OK  ]  86. Fair semaphore: L, R ээлжилнэ
  [OK  ]  87. Хангалттай permit өгөхөд дуусна
  [OK  ]  88. Цуцлахад thread үлдэхгүй

== Командын мөр ==
  [OK  ]  89. Номын жишээ файлаас
  [OK  ]  90. Сондгой n = 11
  [OK  ]  91. Давхардалтай файл
  [OK  ]  92. Урвуу файл
  [OK  ]  93. int-ийн хязгаар файл
  [OK  ]  94. Ганц элемент
  [OK  ]  95. Санамсаргүй өгөгдөл
  [OK  ]  96. Оролтгүй
  [OK  ]  97. Тоо биш утгатай файл
  [OK  ]  98. Хоосон файл
  [OK  ]  99. Буруу алгоритм
  [OK  ] 100. --help

Нийт: 100, амжилттай: 100, алдаатай: 0
Хүснэгт: tests/results/test-results.md
(base) otgo@OtgonaMacBook-Pro F.CSM302-TeamProject-2-4-6 %
```

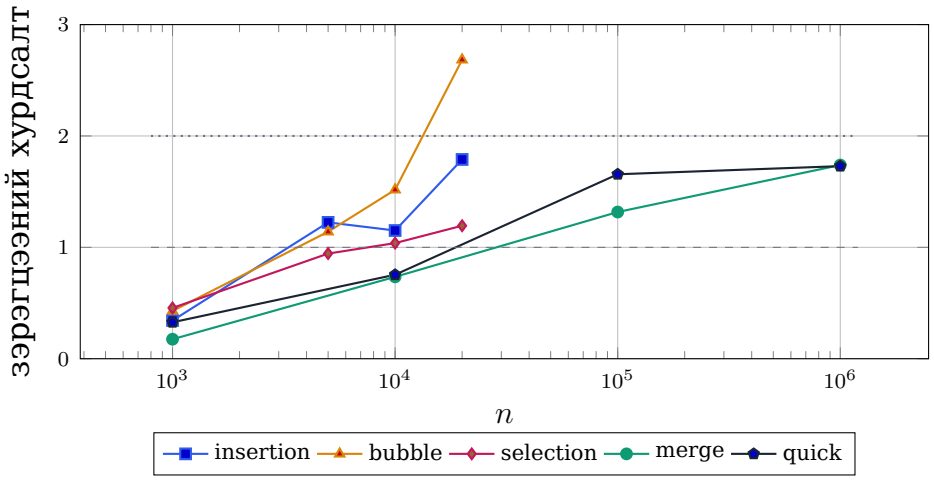
Зураг 8.1. `./test.sh`-ийн гаралтын сүүлийн хэсэг (macOS, Temurin 17)

## 8.2. Хугацааны хэмжилтийн үр дүн

Хүснэгт 8.1-т $n$-ийн хамгийн том утгууд дээрх үр дүнг, Зураг 8.2-т зэрэгцээний хурдсалтыг $n$-ээс хамааруулан харуулав. Бүрэн үр дүн `tests/results/benchmark.md`-д бий.

Хүснэгт 8.1. Хугацаа (ms, медиан). Хаалтанд 3 thread-ийн JVM хоорондын муж

| Алгоритм | $n$ | 1 thread | 1 thr., хувааж | 3 thread | Нэгтгэх үе | Нийт | Зэрэгц. |
|---|---|---|---|---|---|---|---|
| insertion | 20 000 | 180.1 | 90.6 | 50.6 (48.3–54.3) | 0.7 | 3.56 | 1.79 |
| bubble | 20 000 | 466.1 | 198.5 | 73.9 (67.0–145.4) | 0.5 | 6.30 | 2.68 |
| selection | 20 000 | 152.5 | 74.2 | 62.2 (45.9–65.1) | 0.7 | 2.45 | 1.19 |
| merge | 100 000 | 13.9 | 13.7 | 10.4 (9.2–10.4) | 1.1 | 1.34 | 1.32 |
| merge | 1 000 000 | 168.3 | 165.3 | 95.0 (92.8–101.5) | 7.9 | 1.77 | 1.74 |
| quick | 100 000 | 13.7 | 13.6 | 8.2 (8.0–9.4) | 1.0 | 1.66 | 1.66 |
| quick | 1 000 000 | 159.6 | 159.8 | 92.5 (92.3–99.8) | 7.4 | 1.73 | 1.73 |




Зураг 8.2. Зэрэгцээний хурдсалт ($T_{1\,\text{thread, хуваах}}/T_{3\,\text{thread}}$). Тасархай шугам 1 (ашиггүй), цэгэн шугам 2 (2 цөмийн онолын дээд хязгаар)

Жижиг $n$ дээр 3 thread-тэй хувилбар удаан байсан: $n = 100$ үед 1 thread 0.006 ms, 3 thread 0.33 ms (insertion), $n = 1000$ үед merge sort 0.10 ба 0.50 ms. Энэ зөрүү бараг бүхэлдээ thread үүсгэх, `join()` хийх зардал юм. Event trace нь $n = 1000$ үед merge sort-ыг 2.2 дахин (18 694 event), insertion sort-ыг 43.7 дахин (245 187 event) удаашруулсан. Тиймээс хурдыг trace-гүй хэмжсэн.

## 8.3. Илэрсэн ба зассан алдаа

Хөгжүүлэлт, тестийн явцад дараах алдааг илрүүлж зассан: (1) O($n^2$) алгоритмын trace-ийн хязгаар хэт их байсан тул 2 сая event-ийн дээд хязгаарт хүрч, 156 MB файл үүсч байсныг $n \leq 1000$ болгов. (2) Хоосон талбарыг `Integer.MIN_VALUE`-ээр тэмдэглэдэг тул $-2^{31}$ утга trace-ээс алга болж байсныг `long` талбар болгож засав. (3) UI-ийн байрлалын гулсуурыг чирэхэд хуучин байрлал руу буцдаг байсныг засав. (4) Lockstep-ээс Replay руу шилжихэд хуучин session серверт үлддэг байсныг цуцалдаг болгов. (5) Fair semaphore үргэлж L-ээс эхэлнэ гэж үзсэн тест давтан ажиллуулахад унасан тул “ээлжилсэн” эсэхийг шалгадаг болгов. (6) Retina (дэлгэцийн масштаб 2) дэлгэц дээр canvas-ийн өндөр зурах бүрд 2 дахин өсөж, хэдхэн алхмын дараа хөтөч зурж чадахаа больдог байсныг анхны өндрийг нэг удаа уншдаг болгож засав.

# 9. Хэлэлцүүлэг

## 9.1. Үр дүнгийн тайлбар

Merge ба quick sort-д “1 thread” ба “1 thread, хуваж” хугацаа бараг тэнцүү. Учир нь хоёр хэсэгт хуваах нь ажлыг багасгадаггүй: $2 \cdot \frac{n}{2} \log \frac{n}{2} \approx n \log n$. Иймээс 1.73–1.77 хурдсалт бүхэлдээ зэрэгцээнээс ирж байна. $n = 10^6$ үед merge sort-ын нэгтгэх үе 7.9 ms бөгөөд энэ нь ажлын $S \approx 7.9/165.3 \approx 0.048$ буюу 4.8 %. Амдалын хуулиар (3.1) $N = 2$ үед дээд хязгаар $1/(0.048 + 0.952/2) \approx 1.91$. Хэмжсэн 1.74 нь үүнээс бага бөгөөд зөрүүг thread үүсгэх зардал болон хоёр цөм санах ойн зурвасыг (memory bandwidth) хуваалцдагтай холбож тайлбарлаж байна.

O($n^2$) алгоритмд хуваалт өөрөө ажлыг 2 дахин багасгадаг: $2 \cdot (n/2)^2 = n^2/2$ (insertion 180 → 91 ms). Үүн дээр зэрэгцээний ашиг нэмэгдэж, онолын нийт хурдсалт 4 болно. Insertion sort-ын 3.56 үүнд ойрхон байна. Иймээс “O($n^2$) алгоритмд 2 thread 3.5 дахин хурдан” гэж дүгнэх нь төөрөгдөл бөгөөд ашгийн тал орчим нь алгоритмын шинж чанараас ирж байгаа юм. Олон урсгалт хувилбар merge/quick-д $n \approx 10^5$-аас, O($n^2$)-д $n \approx 5000$-аас эхэлж ашигтай болсон. Үүнээс бага $n$-д thread-ийн зардал давамгайлна. Жишээ нь номын жишээн дээр Sorter-L нь Sorter-R эхлэхээс өмнө дуусчихдаг. Энэ нь алдаа биш, thread үүсгэх нь 5 элемент эрэмбэлэхээс удаан байдагтай холбоотой.

Lockstep горимд хоёр sorter L, R, L, R гэж хатуу ээлжилсэн. Эхэндээ үүнийг OS-ийн scheduler шийддэг гэж үзсэн боловч шалтгаан нь fair semaphore-ийн FIFO дараалал байв. Харин эхний permit-ийг аль sorter авахыг scheduler шийддэг.

## 9.2. Хязгаарлалт

Хэмжилтийг 2 цөмтэй виртуал машин дээр хийсэн тул JIT ба GC-ийн туслах thread-үүд sorter-уудтай цөмийн төлөө өрсөлддөг. Bubble sort-ын 3 thread-ийн хугацаа JVM-ээс хамаарч 67–145 ms хооронд хэлбэлзсэн бөгөөд түүний зэрэгцээний хурдсалт 2.68 нь 2 цөмийн физик хязгаараас давсан. Иймд энэ утгыг JIT-ийн нөлөө гэж үзэж итгэл муутай гэж тооцсон. 1 ms-ээс бага хугацаа шуугиантай. Windows-ийн `.bat` скриптийг бодит Windows машин дээр туршаагүй.

## 9.3. Сайжруулах боломж

$k$ sorting thread бүхий мод хэлбэрийн нэгтгэл нь олон цөмийг бүрэн ашиглана. Нэгтгэх үеийг хоёр thread-д хуваавал (нэг нь эхнээс, нөгөө нь төгсгөлөөс) Амдалын $S$ багасна. Thread pool болон босго утга (жижиг $n$-д дан thread) ашиглавал жижиг өгөгдлийн зардал буурна. `Arrays.parallelSort` ч мөн ийм босго ашигладаг [6].

## 9.4. GitHub ба AI ашигласан тухай

`22dbddf` ба `a8916d6` commit-ууд ижил мессежтэй. Учир нь web UI-г шалгалт дуусахаас өмнө commit хийсэн тул засвар нь дараагийн commit-д орсон. Хуваалцсан түүхийг `force push`-оор өөрчлөхгүйн тулд хэвээр үлдээв.

Энэхүү төслийн код, тест, баримт бичиг болон тайлангийн нооргийг бэлтгэх, алгоритмын ажиллагааг тайлбарлах, алдаа засварлахад Claude хиймэл оюун ухааныг ашигласан. Багийн гишүүд бүх материалыг уншиж, шалган засварлаж, програмыг өөрсдийн компьютер дээр ажиллуулж баталгаажуулсан. Бүх `git` үйлдлийг багийн гишүүд өөрсдөө хийсэн бөгөөд тайлан дахь тоо бүр програмыг бодитоор ажиллуулж гарсан утга юм.

# 10. Дүгнэлт

“Multithreaded Sorting Application” төслийг Java 17 дээр бүрэн хэрэгжүүлэв. Програм жагсаалтыг хоёр хуваан хоёр sorting thread-ээр параметрээр өгсөн мужийг эрэмбэлж, merging thread-ээр хоёр дахь global массив руу нэгтгэнэ. Parent thread бүх thread-ийг `join()` хийсний дараа үр дүнг хэвлэнэ. Үүн дээр таван алгоритм, оролтын бүрэн шалгалт, JSON event trace, бодит thread-тэй холбогдсон Sorting animation (Replay ба Lockstep горим) хийж, 100 автомат тестээр шалгасан.

Ажлын гол сургамж нь гурав. Нэгдүгээрт, `start()`/`join()` нь өөрөө синхрончлол бөгөөд давхцахгүй муж ба join-ийн дарааллын ачаар lock огт хэрэггүй. Хоёрдугаарт, зэрэгцээ нь үнэгүй биш: жижиг $n$-д thread-ийн зардал давамгайлж, их $n$-д ч нэгтгэх дан үеэс болж хурдсалт 2-т хүрэхгүй (Амдалын хууль). Гуравдугаарт, хурдсалтыг зөв тайлбарлахын тулд “1 thread, хувааж” гэх мэт тохирох суурь хэмжилт болон олон JVM-ийн медиан шаардлагатай. Цаашид олон thread бүхий мод хэлбэрийн нэгтгэл, зэрэгцээ merge ашиглаж олон цөмийг илүү бүрэн ашиглах боломжтой.

# Ашигласан материал

[1] A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018. Chapter 4 “Threads & Concurrency”, Chapter 6 “Synchronization Tools”.

[2] G. M. Amdahl. “Validity of the single processor approach to achieving large scale computing capabilities.” In *Proc. AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.

[3] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, G. Bierman. *The Java Language Specification, Java SE 17 Edition*, Chapter 17 “Threads and Locks”. Oracle, 2021. https://docs.oracle.com/javase/specs/jls/se17/html/jls-17.html

[4] Oracle. *Java SE 17 API: `java.lang.Thread`, `Thread.State`*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html

[5] Oracle. *Java SE 17 API: `java.util.concurrent.Semaphore`*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/Semaphore.html

[6] Oracle. *Java SE 17 API: `java.util.Arrays.parallelSort`*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Arrays.html

[7] B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes, D. Lea. *Java Concurrency in Practice*. Addison-Wesley, 2006.

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[9] C. A. R. Hoare. “Quicksort.” *The Computer Journal*, 5(1):10–16, 1962.

[10] Багийн GitHub repository. https://github.com/Munkhgerelt/F.CSM302-TeamProject-2-4-6